

Taller 4: Planificación Automatizada

2. Análisis del algoritmo (forwardBFS)

Se implementó un algoritmo de búsqueda en anchura (forwardBFS) para resolver problemas de planificación en el dominio de rescate. El algoritmo explora estados desde el estado inicial hasta encontrar uno que satisfaga el objetivo del problema.

```
PS C:\Users\Win\Downloads\taller4_IA\PhoenixOperation> python main.py -p SimpleRescueProblem -f forwardBFS -l openRescue -q

=====
Operación Fénix - ISIS-1611 Inteligencia Artificial
=====
Layout: openRescue (12x10)
Problema: SimpleRescueProblem
Pacientes: ['patient_0']
Suministros: ['supplies_0']
Puestos médicos: [(4, 2)]
=====
Planificador: forwardBFS

Tiempo de planificación: 9.458s
Estados expandidos: 9358
Longitud del plan: 26 acciones

Plan:
  1. Move(robot, (1, 8), (1, 7))
  2. Move(robot, (1, 7), (1, 6))
```

```
PS C:\Users\Win\Downloads\taller4_IA\PhoenixOperation> python main.py -p SimpleRescueProblem -f forwardBFS -l tinyBase -q

=====
Operación Fénix - ISIS-1611 Inteligencia Artificial
=====
Layout: tinyBase (5x7)
Problema: SimpleRescueProblem
Pacientes: ['patient_0']
Suministros: ['supplies_0']
=====
Planificador: forwardBFS

Tiempo de planificación: 0.011s
Estados expandidos: 225
Longitud del plan: 9 acciones

Plan:
  1. Move(robot, (1, 4), (1, 3))
  2. PickUp(robot, supplies_0, (1, 3))
  3. Move(robot, (1, 3), (1, 2))
  4. GetUpSupplies(robot, supplies_0, (1, 2))
```

Las pruebas se realizaron sobre los layouts tinyBase y openRescue.

Resultados obtenidos

Layout	Estados expandidos	Longitud del plan	Tiempo
tinyBase	225	9 acciones	0.011s
openRescue	9358	26 acciones	9.458s

Observaciones

En el layout tinyBase, el algoritmo encontró rápidamente un plan debido al tamaño reducido del mapa y al bajo número de acciones posibles.

En contraste, en openRescue el número de estados expandidos aumentó considerablemente. Esto ocurre porque BFS explora sistemáticamente todas las posibilidades por niveles, y al crecer el mapa aumentan también las combinaciones posibles de movimientos y acciones.

A pesar del incremento en el espacio de búsqueda, el algoritmo logró encontrar un plan válido y completar exitosamente la misión en ambos escenarios.

3. Analisis del algoritmo backwardSearch.

```
PS C:\Users\Juank\Documents\AI\taller 4\taller4_IA\PhoenixOperation> py main.py -p SimpleRescueProblem -f backwardSearch -l tinyBase -q

=====
Operación Fénix - ISIS-1611 Inteligencia Artificial
=====
Layout: tinyBase (5x7)
Problema: SimpleRescueProblem
Pacientes: ['patient_0']
Suministros: ['supplies_0']
Puestos médicos: [(1, 2)]
=====
Planificador: backwardSearch

Tiempo de planificación: 0.096s
Estados expandidos: 936
Longitud del plan: 9 acciones

Plan:
1. Move(robot, (1, 4), (1, 3))
2. PickUp(robot, supplies_0, (1, 3))
3. Move(robot, (1, 3), (1, 2))
4. SetupSupplies(robot, supplies_0, (1, 2))
5. Move(robot, (1, 2), (1, 1))
6. PickUp(robot, patient_0, (1, 1))
7. Move(robot, (1, 1), (1, 2))
8. PutDown(robot, patient_0, (1, 2))

Ejecutando plan...

¡Misión completada exitosamente!
```

d) Se espera que en la búsqueda regresiva sea más eficiente en problemas donde el objetivo final tiene pocos flujos y las acciones relevantes para alcanzarlos son fáciles de identificar. Es decir, cuando hay muchos estados iniciales posibles o muchas acciones forward irrelevantes, pero pocos caminos útiles hacia el objetivo. En cambio, la búsqueda hacia adelante puede ser mejor cuando casi todas las acciones disponibles desde el inicio

contribuyen al objetivo, o cuando la regresión genera demasiados subobjetivos alternativos. En el dominio de rescate, backward search puede reducir el factor de ramificación porque parte de Rescued(patient) y solo considera acciones que contribuyen a ese objetivo, aunque los movimientos pueden aumentar mucho el espacio de objetivos regresados.

4. Analisis de heurísticas

```
PS C:\Users\Juan\Documents\AI\taller 4\taller4_IA\PhoenixOperation> py main.py -p SimpleRescueProblem -f aStarPlanner -h ignorePreconditions -l tinyBase -q

=====
Operación Fénix - ISIS-1611 Inteligencia Artificial
=====
Layout:   tinyBase (5x7)
Problema: SimpleRescueProblem
Pacientes: ['patient_0']
Suministros: ['supplies_0']
Puestos médicos: [(1, 2)]
=====
Planificador: aStarPlanner
Heurística: ignorePreconditions

Tiempo de planificación: 0.923s
Estados expandidos: 252
Longitud del plan: 9 acciones

Plan:
1. Move(robot, (1, 4), (1, 3))
2. Pickup(robot, supplies_0, (1, 3))
3. Move(robot, (1, 3), (1, 2))
4. SetupSupplies(robot, supplies_0, (1, 2))
5. Move(robot, (1, 2), (1, 1))
6. Pickup(robot, patient_0, (1, 1))
7. Move(robot, (1, 1), (1, 2))
8. PutDown(robot, patient_0, (1, 2))
9. Rescue(robot, patient_0, (1, 2))

Ejecutando plan...
¡Misión completada exitosamente!
```

```
PS C:\Users\Juan\Documents\AI\taller 4\taller4_IA\PhoenixOperation> py main.py -p SimpleRescueProblem -f aStarPlanner -h ignoreDeleteLists -l tinyBase -q

=====
Operación Fénix - ISIS-1611 Inteligencia Artificial
=====
Layout:   tinyBase (5x7)
Problema: SimpleRescueProblem
Pacientes: ['patient_0']
Suministros: ['supplies_0']
Puestos médicos: [(1, 2)]
=====
Planificador: aStarPlanner
Heurística: ignoreDeleteLists

Tiempo de planificación: 0.170s
Estados expandidos: 21
Longitud del plan: 9 acciones

Plan:
1. Move(robot, (1, 4), (1, 3))
2. Pickup(robot, supplies_0, (1, 3))
3. Move(robot, (1, 3), (1, 2))
4. SetupSupplies(robot, supplies_0, (1, 2))
5. Move(robot, (1, 2), (1, 1))
6. Pickup(robot, patient_0, (1, 1))
7. Move(robot, (1, 1), (1, 2))
8. PutDown(robot, patient_0, (1, 2))
9. Rescue(robot, patient_0, (1, 2))

Ejecutando plan...
¡Misión completada exitosamente!
```

4A. b) En el layout tinyBase, el objetivo real es lograr ("Rescued", "patient_0"), y el plan válido necesita 9 acciones porque primero hay que recoger suministros, llevarlos al puesto médico, recoger al paciente, llevarlo al puesto y ejecutar Rescue. Pero con la heurística ignorePreconditionsHeuristic, se ignoran todas las precondiciones, así que la acción Rescue(robot, patient_0, (1,2)) puede considerarse aplicable inmediatamente y alcanza el objetivo en 1 acción. Entonces la heurística estima $h(n) = 1$, mientras que el costo real es 9,

por lo tanto $1 \leq 9$; no sobreestima. Esto muestra que es admisible, porque resuelve una versión relajada y más fácil del problema original.

4B. d) La heurística más informativa en el dominio de rescate es ignoreDeleteListsHeuristic. La evidencia empírica en tinyBase fue que A* con ignorePreconditionsHeuristic expandió 252 estados, mientras que A* con ignoreDeleteListsHeuristic expandió solo 21 estados, encontrando en ambos casos un plan válido de 9 acciones.

Analisis de HTN

Complejidad en espacio

El algoritmo mantiene en memoria la cola BFS de planes parciales. Cada plan es una secuencia de hasta d pasos, y en cada nodo se pueden generar hasta r refinamientos (donde r es el número máximo de refinamientos por HLA). En el peor caso la cola crece exponencialmente: $O(r^d)$ planes, cada uno de longitud d , dando una complejidad espacial de $O(d \cdot r^d)$. El conjunto *visited_plans* también crece con el número de planes explorados, contribuyendo el mismo orden.

Complejidad en tiempo

En el peor caso se exploran todos los planes posibles hasta profundidad d , con r refinamientos por HLA. Esto da $O(r^d)$ nodos expandidos. Para cada plan primitivo se ejecuta una simulación de longitud d para verificar si alcanza el goal, añadiendo un factor d , resultando en $O(d \cdot r^d)$ en el peor caso.

Complejidad

El algoritmo es completo bajo las siguientes condiciones: que el espacio de refinamientos sea finito, que cada HLA tenga al menos un refinamiento, y que *visited_plans* no descarte planes válidos. En nuestra implementación usamos *id()* de los objetos como clave, lo que evita descartar planes estructuralmente distintos. Si existe un plan primitivo que alcanza el goal, BFS lo encontrará eventualmente.

Optimalidad

El algoritmo no garantiza optimalidad en general. BFS sobre el espacio de planes jerárquicos encuentra el plan con menor número de refinamientos aplicados, pero no necesariamente el plan primitivo más corto. En nuestra implementación específica esto no es un problema grave porque cada HLA tiene exactamente un refinamiento concreto generado por BFS sobre el mapa, por lo que el plan resultante es de longitud óptima dentro de la jerarquía definida. Sin embargo, si hubiera múltiples refinamientos alternativos para una misma HLA, BFS encontraría el de menor profundidad jerárquica, no necesariamente el de menor número de acciones primitivas.

Admisibilidad, consistencia y ventaja sobre planificación clásica:

El planificador HTN no usa heurísticas en el sentido clásico, así que las preguntas de admisibilidad y consistencia no aplican directamente. Sin embargo:

La ventaja principal del HTN sobre la planificación clásica está en la reducción del factor de ramificación. En forward BFS clásico, desde cada estado se consideran todas las acciones primitivas aplicables, que pueden ser $O(a)$ donde a es el número de acciones instanciadas. En un mapa de 9×8 con todas las celdas libres, solo las acciones **Move** pueden ser decenas, más todas las combinaciones de PickUp, PutDown, etc.

En HTN, el espacio de búsqueda está estructurado por la jerarquía: en lugar de explorar todas las acciones posibles, solo se consideran los refinamientos válidos de cada HLA, que en nuestra implementación son secuencias fijas generadas por BFS. Esto reduce drásticamente el espacio efectivo de búsqueda.

Formalmente, si el planificador clásico explora $O(a^d)$ nodos, el HTN explora $O(r^d)$ donde $r \ll a$ porque los refinamientos encapsulan conocimiento del dominio. En el caso extremo de nuestra implementación, cada HLA tiene exactamente un refinamiento, haciendo la búsqueda casi lineal en la profundidad jerárquica

6. Analisis de IA

En el desarrollo de este taller, el apoyo de la IA fue entendido como una herramienta de co-construcción y no como un reemplazo del proceso de razonamiento propio. En primer lugar, se intentó construir una versión inicial de las soluciones a partir de la comprensión del enunciado, de los docstrings y de la lógica general del proyecto. A partir de esa base, la IA se utilizó principalmente para revisar si las ideas iniciales estaban bien encaminadas, ayudar a detectar errores de lógica, mejorar la claridad del código y proponer formas un poco más limpias de escribir algunas funciones. En ese sentido, el aprendizaje principal fue que la IA resulta mucho más útil cuando ya existe una idea previa del problema, porque permite contrastar decisiones, encontrar detalles que uno pasa por alto y entender mejor por qué una implementación funciona o por qué puede fallar en ciertos casos.

En cuanto al tipo de correcciones realizadas, en la mayoría de los casos no se trató de cambios que transformaran completamente la solución, sino de ajustes más bien secundarios: simplificación de estructuras, mejoras de legibilidad, organización de variables, detección de posibles casos problemáticos y una mejor explicación de la diferencia entre funciones que, a primera vista, parecían similares.